

PROJECT MS

신규 캐릭터 개발 가이드

공통 구조를 이해하고 스킬과 패시브 구현에 집중하는
Photon Fusion 2 Shared Mode 제작 가이드

대단원별 페이지 분리 | 세부 내용은 한 흐름으로 연결

한눈에 보는 캐릭터 실행 흐름

위에서 아래로 읽으면 캐릭터 행동 하나가 입력되고, 실행되고, 동기화되어 화면에 표시되는 과정을 볼 수 있습니다.



개발자가 작업하는 곳

3단계의 On... 함수에 캐릭터 규칙을 작성하고, 그 안에서 4단계의 공통 API를 호출합니다. 나머지 단계는 공통 시스템이 처리합니다.

문서 목적

이 문서는 현재 캐릭터 공통 프레임워크의 구조와 신규 캐릭터를 만드는 절차만 설명한다.

대상: 신규 캐릭터의 스킬과 패시브를 구현하는 개발자

네트워크 기준: Photon Fusion 2, GameMode.Shared

목표: 캐릭터 제작자는 Fusion 내부 구현보다 캐릭터의 전투 규칙에 집중한다.

범위: 공통 구조, 프리팹 구성, 데이터 설정, Character API, 스킬과 패시브 구현, 검수 절차

1. 전체 구조와 역할 분리

캐릭터 행동 한 번은 입력 수집부터 화면 표현까지 아래 6단계로 처리된다.

단계	실행 흐름	담당	신규 캐릭터 개발자의 작업
1	버튼과 조준 방향 수집	CharacterInputHandler	없음
2	권한, 생존, 쿨타임 검사	CharacterBase	없음
3	해당 행동 함수 호출	OnBasicAttack, OnSkillQ, OnSkillE, OnUltimate, OnDash	필요한 함수에 전투 규칙 작성
4	판정과 결과 요청	FindEnemies..., DealDamage, SpawnProjectile, StartDash	구현된 공통 API를 호출
5	체력, 쿨타임, 행동 상태 동기화	CharacterBase와 Fusion	없음
6	이동, 피격, 스킬 효과 표시	CharacterVisualController	프리팸과 Profile 연결

신규 캐릭터 개발자가 코드를 작성하는 핵심 지점은 3단계다. 그 코드 안에서 4단계의 공통 API를 호출한다. 1, 2, 5단계의 네트워크 흐름은 Base가 처리하고, 6단계는 공통 비주얼 시스템이 처리한다.

역할 분리

영역	책임	신규 캐릭터 개발자가 수정하는가?
CharacterBase	네트워크 실행 흐름, 공통 전투 API, 상태 동기화	아니요
CharacterInputHandler	입력 수집과 짧은 입력 보관	아니요
CharacterMovementHandler	이동, 점프, 대시, 지면 판정	보통 수정하지 않음
CharacterHealth	체력과 사망 처리	아니요
CharacterCooldownHandler	행동별 쿨타임 처리	아니요
CharacterVisualController	공통 움직임과 피격 표현	설정만 연결
캐릭터별 클래스	평타, Q, E, 궁극기, 패시브	예
CharacterDefinition	체력, 이동, 데미지, 쿨타임 수치	예
CharacterVisualProfile	공통 비주얼 반응 수치	필요한 경우만

2. 폴더와 주요 파일

```

Character
├── Runtime
│   ├── Core
│   │   ├── CharacterBase.cs
│   │   ├── CharacterActionContext.cs
│   │   └── CharacterSockets.cs
│   ├── Data
│   │   ├── CharacterDefinition.cs
│   │   └── CharacterVisualProfile.cs
│   ├── Modules
│   │   ├── CharacterInputHandler.cs
│   │   ├── CharacterMovementHandler.cs
│   │   ├── CharacterHealth.cs
│   │   └── CharacterCooldownHandler.cs
│   ├── Combat
│   │   ├── CharacterCombatQuery2D.cs
│   │   └── CharacterProjectile.cs
│   ├── Visual
│   │   ├── CharacterVisualController.cs
│   │   └── Solver
│   └── Examples
│       └── CharacterTemplate.cs
├── Editor
│   └── CharacterPrefabValidator.cs
└── Docs
  
```

신규 캐릭터를 만들 때 주로 다루는 것은 캐릭터별 스크립트, **CharacterDefinition**, 프리팹 Variant 세 가지다. 공통 Core, Modules, Combat, Visual 코드는 공통 시스템 담당자와 협의하지 않고 직접 변경하지 않는다.

3. 캐릭터 프리팹 구조

모든 캐릭터는 동일한 기본 프리팹 구조를 사용한다.

```

CharacterRoot
├── 캐릭터별 스크립트 : CharacterBase 상속
├── Rigidbody2D
├── Collider2D
├── NetworkObject
├── 프로젝트 표준 NetworkRigidbody2D
├──
├── VisualRoot
│   ├── CharacterVisualController
│   ├── Body
│   ├── AttackOrigin
│   ├── ProjectileOrigin   선택
│   ├── WeaponRoot       선택
│   ├── EffectOrigin      선택
│   ├── WorldCanvas       선택
│   └── DustEffect         선택

```

Socket 사용 기준

여기서 Socket은 통신에 사용하는 네트워크 Socket이 아니다. 캐릭터마다 다른 몸 크기, 손 위치, 무기 위치를 공통 코드에 전달하기 위해 프리팹에 배치하는 빈 **Transform** 기준점이다. 별도의 스크립트나 네트워크 컴포넌트를 추가하지 않는다.

Socket	역할과 미지정 시 처리	예시
Attack Origin	근접 공격, 범위 판정, 조준 계산의 기준 위치. 비어 있으면 CharacterRoot 사용	검, 주먹, 부채꼴 공격
Projectile Origin	투사체 생성 위치. 비어 있으면 Attack Origin 사용	총알, 화살, 마법탄
Weapon Root	무기 비주얼을 붙이는 부모 Transform. 비어 있으면 사용하지 않음	검, 총, 활, 지팡이
Effect Origin	공격 VFX의 기본 위치. 비어 있으면 Attack Origin 사용	폭발, 섬광, 타격 효과

CharacterActionContext.Origin은 내부적으로 **AttackOrigin.position**을 사용한다. **SpawnProjectile**에는 **ProjectileOrigin.position**, 공격 VFX에는 **EffectOrigin.position**을 전달한다. 따라서 신규 캐릭터 개발자는 Socket 오브젝트에 기능을 구현하는 것이 아니라, 프리팹에서 필요한 기준점의 위치만 맞춘다.

사용하지 않는 Socket은 비워둘 수 있다. 검 캐릭터에 **ProjectileOrigin**을 억지로 만들거나, 무기가 없는 캐릭터에 **WeaponRoot**를 추가할 필요는 없다. 다만 Attack Origin은 비워도 CharacterRoot로 대체되지만 정확한 공격 판정을 위해 캐릭터 몸 앞의 적절한 위치에 연결하는 것을 권장한다. Socket에는 컴포넌트를 붙이지 않으며, 공통 움직임과 피격 표현은 **CharacterVisualController**에서 설정한다.

4. 캐릭터 데이터 만들기

Character Definition

Unity Project 창에서 다음 메뉴로 생성한다.

Create > Project MS > Character > Definition

CharacterDefinition에는 캐릭터의 기본 수치를 설정한다.

표시 이름과 최대 체력

이동 속도, 가속도, 점프 힘

지면 판정 Layer

점프 버퍼와 코요테 타임

대시 힘과 지속 시간

Auto Hop 사용 여부와 수치

평타, Q, E, 궁극기 데미지와 쿨타임

기본 입력 키

실행 중 변하는 현재 체력, 남은 쿨타임, 버프 상태는 Definition에 저장하지 않는다.

Character Visual Profile

기본적으로 팀 공통 Profile을 사용한다. 캐릭터의 무게감이나 탄성이 확실히 달라야 할 때만 별도 Profile을 생성한다.

Create > Project MS > Character > Visual Profile

Profile에서 조정할 수 있는 항목은 점프 Stretch, 착지 Squash, 대기 Wobble, 피격 플래시, Jiggle 복원력과 감쇠 등이다.

5. 캐릭터별 스크립트 구현

캐릭터 클래스에는 직접 구현할 함수와 이미 구현된 공통 API가 함께 보인다. 두 종류를 구분하는 것이 가장 중요하다.

5.1 구현 상태 구분

구분	의미	캐릭터 개발자의 작업
직접 구현	Base에는 실행 위치만 있고 캐릭터 규칙은 비어 있음	필요한 함수에서 override 작성
선택 구현	기본 동작 또는 빈 이벤트가 이미 있음	캐릭터 특성이 필요할 때만 override
호출만 사용	공통 기능의 실제 코드가 Base에 구현되어 있음	스킬 함수 안에서 호출
Base 자동 처리	입력, 권한, 쿨타임, 동기화 처리	작성하거나 직접 호출하지 않음

캐릭터가 직접 구현하는 함수

함수	구현 여부	Base의 기본 상태
OnBasicAttack	평타가 있으면 직접 구현	false 반환
OnSkillQ	Q를 사용하면 직접 구현	false 반환
OnSkillE	E를 사용하면 직접 구현	false 반환
OnUltimate	궁극기를 사용하면 직접 구현	false 반환
OnDash	특수 대시일 때만 구현	기본 대시 실행 후 true 반환
OnDamaged 등 패시브 이벤트	해당 패시브가 있을 때만 구현	아무 동작도 하지 않음

OnBasicAttack, OnSkillQ, OnSkillE, OnUltimate는 C# 문법상 필수 override는 아니다. 하지만 해당 입력으로 스킬을 사용하려면 반드시 구현해야 한다. 구현하지 않은 함수는 false를 반환하므로 아무 행동도 실행되지 않고 쿨타임도 시작되지 않는다.

일반적인 좌우 대시는 OnDash를 구현하지 않아도 된다. CharacterBase.OnDash가 이미 StartDefaultDash()를 호출한다. 조준 방향 돌진이나 후방 회피처럼 기본 대시와 다른 캐릭터만 OnDash를 override한다.

이미 구현되어 호출만 하는 함수

기능	호출할 공통 API
대상 검색	FindEnemiesInCircle, FindEnemiesInBox, FindEnemiesInLine, FindEnemiesInArc
피해와 회복	DealDamage, Heal
투사체 생성	SpawnProjectile
이동	StartDefaultDash, StartDash
공통 VFX	PlayActionEffect

위 함수들의 판정, 권한 확인, NetworkObject 생성 같은 내부 코드는 이미 구현되어 있다. 캐릭터 개발자는 이 함수를 다시 만들거나 override하지 않고 자신의 스킬 함수 안에서 호출한다.

Base가 자동으로 처리하는 부분

입력을 읽고 해당 스킬 진입점을 호출한다.

실행 전에 남은 쿨타임을 검사한다.

함수가 true를 반환했을 때만 쿨타임을 시작한다.

동기화할 행동 상태와 Sequence를 갱신한다.

성공한 행동에 대해 OnSkillExecuted를 호출한다.

RPC, StateAuthority, InputAuthority와 Runner.Spawn을 내부에서 처리한다.

5.2 기본 캐릭터 템플릿

CharacterTemplate.cs를 복사하고 클래스와 파일 이름을 캐릭터 이름으로 변경한다. 캐릭터가 사용하는 스킬 함수만 override한다.

```

public sealed class NewCharacter : CharacterBase
{
    protected override bool OnBasicAttack(CharacterActionContext context)
    {
        // 평타 규칙
        return true;
    }

    protected override bool OnSkillQ(CharacterActionContext context)
    {
        // Q 스킬 규칙
        return true;
    }

    protected override bool OnSkillE(CharacterActionContext context)
    {
        // E 스킬 규칙
        return true;
    }

    protected override bool OnUltimate(CharacterActionContext context)
    {
        // 궁극기 규칙
        return true;
    }

    // 일반 대시는 OnDash를 작성하지 않아도 동작한다.
}

```

스킬이 정상적으로 실행되었으면 **true**, 실행 조건이 맞지 않아 취소되었으면 **false**를 반환한다. 공통 흐름은 성공한 행동에 대해서만 쿨타임과 후속 이벤트를 처리한다.

5.3 공격 방식에 맞는 공통 API 선택

무기 종류에 따라 상속 구조를 다시 만들지 않는다. 스킬 안에서 판정 API만 선택한다.

구현하려는 동작	사용하는 API
총알, 화살, 마법탄	SpawnProjectile
검이나 부채꼴 근접 공격	FindEnemiesInArc + DealDamage
자신 중심 범위 공격	FindEnemiesInCircle + DealDamage
직선 관통 공격	FindEnemiesInLine + DealDamage
사각형 범위 공격	FindEnemiesInBox + DealDamage
기본 대시	StartDefaultDash
방향과 힘이 다른 대시	StartDash
공통 공격 VFX	PlayActionEffect

CharacterActionContext에서 조준 방향, 조준 각도, 공격 기준 위치, 데미지 등 현재 행동에 필요한 값을 가져온다.

5.4 패시브 구현

패시브는 매 프레임 직접 검사하지 않고 상황에 맞는 이벤트를 우선 사용한다.

발생 조건	override 함수
피격됨	OnDamaged
다른 캐릭터에게 피해를 줌	OnDamageDealt
사망함	OnDied
점프함	OnJumped
착지함	OnLanded
스킬 실행에 성공함	OnSkillExecuted
체력이 변경됨	OnHealthChanged
지속적으로 확인해야 함	OnPassiveTick

예를 들어 피격 시 방어력이 오르는 패시브는 OnDamaged, 착지 시 충격파가 발생하는 패시브는 OnLanded에 구현한다. 이벤트로 표현할 수 없는 지속 조건에만 OnPassiveTick을 사용한다.

6. 캐릭터 제작용 API 안내

이 절에 있는 API는 `CharacterBase`를 상속한 캐릭터 클래스에서 사용한다. 네트워크 권한과 실행 위치는 Base가 처리하므로 캐릭터 코드는 아래 API의 게임 규칙만 작성한다.

문서 표시	구현 상태
직접 구현	캐릭터 클래스에서 override
선택 구현	필요한 캐릭터만 override
호출만	<code>CharacterBase</code> 에 구현 완료, 스킬에서 호출
조회만	Base가 제공하는 현재 상태를 읽기만 함

6.1 직접 구현: 스킬 진입점

입력과 쿨타임 검사를 통과하면 Base가 해당 함수를 호출한다.

API	구현 상태	호출 시점	Base 기본 동작
<code>OnBasicAttack(context)</code>	직접 구현	기본 공격 입력	false 반환
<code>OnSkillQ(context)</code>	직접 구현	Q 입력	false 반환
<code>OnSkillE(context)</code>	직접 구현	E 입력	false 반환
<code>OnDash(context)</code>	선택 구현	대시 입력	기본 대시 후 true
<code>OnUltimate(context)</code>	직접 구현	궁극기 입력	false 반환

행동이 실제로 실행되었으면 `true`를 반환한다. 거리나 자원 같은 추가 조건이 맞지 않아 행동을 취소했으면 `false`를 반환한다. `false`를 반환한 행동에는 공통 쿨타임과 성공 이벤트가 적용되지 않는다.

6.2 Base 제공 값: `CharacterActionContext`

모든 스킬 진입점은 현재 행동에 필요한 값을 `context`로 받는다.

값	타입	의미
context.Action	CharacterActionType	현재 실행 중인 행동 종류
context.Origin	Vector2	공동 시스템이 결정한 공격 기준 위치
context.AimDirection	Vector2	정규화된 조준 방향
context.AimWorldPosition	Vector2	조준 중인 월드 좌표
context.AimAngle	float	조준 방향을 각도로 변환한 값
context.Damage	float	Definition에 설정한 현재 행동의 기본 데미지

캐릭터 스크립트에서 마우스 위치나 입력을 다시 읽지 않고 **context** 값을 사용한다. 그래야 로컬과 원격에서 같은 방향과 결과가 사용된다.

6.3 호출만: 타깃 검색 API

검색 API는 자기 자신을 제외하고 **targetLayer**에 포함된 캐릭터 목록을 반환한다. 검색 결과에 데미지를 주려면 각 대상에 **DealDamage**를 호출한다.

API	영역	주요 인자
FindEnemiesInCircle	원형 범위	center, radius, targetLayer
FindEnemiesInBox	회전 가능한 사각 범위	center, size, angle, targetLayer
FindEnemiesInLine	폭이 있는 직선 범위	origin, direction, distance, width, targetLayer
FindEnemiesInArc	방향을 가진 부채꼴 범위	origin, direction, radius, angle, targetLayer

FindEnemiesInCircle

캐릭터 중심 폭발, 오라, 착지 충격파처럼 방향이 필요 없는 범위 판정에 사용한다.

```
List targets = FindEnemiesInCircle(
    transform.position,
    radius,
    targetLayer);
```

FindEnemiesInBox

벽, 넓은 검기, 직사각형 장판처럼 가로와 세로 크기가 다른 판정에 사용한다. **angle**은 사각형의 회전 각도다.

```
List targets = FindEnemiesInBox(
    context.Origin,
    boxSize,
    context.AimAngle,
    targetLayer);
```

FindEnemiesInLine

레이저나 직선 관통 공격처럼 시작점부터 일정 거리까지 이어지는 판정에 사용한다. **width**로 선의 두께를 지정한다.

```
List targets = FindEnemiesInLine(
    context.Origin,
    context.AimDirection,
    distance,
    width,
    targetLayer);
```

FindEnemiesInArc

검, 주먹, 부채꼴 브레스처럼 바라보는 방향을 중심으로 펼쳐지는 공격에 사용한다. **radius**는 사거리, **angle**은 전체 부채꼴 각도다.

```
List targets = FindEnemiesInArc(
    context.Origin,
    context.AimDirection,
    radius,
    angle,
    targetLayer);
```

6.4 호출만: 전투 API

DealDamage

DealDamage(target, amount)는 대상에게 피해를 요청하고 공격자의 **OnDamageDealt** 이벤트를 호출한다. 대상이 없거나 자기 자신이거나 데미지가 0 이하이면 적용하지 않는다.

```
foreach (CharacterBase target in targets)
{
    DealDamage(target, context.Damage);
}
```

Heal

Heal(amount)는 자신의 체력을 회복한다. 실제 체력 변경과 권한 처리는 Base가 담당한다.

```
Heal(healAmount);
```

SpawnProjectile

투사체 NetworkObject를 한 번만 생성하고 방향, 속도, 데미지, 대상 Layer를 초기화한다.

```
SpawnProjectile(
    projectilePrefab,
    ProjectileOrigin.position,
    context.AimDirection,
    projectileSpeed,
    context.Damage,
    targetLayer);
```

인자는 순서대로 투사체 프리팹, 생성 위치, 방향, 속도, 데미지, 공격 대상 Layer다. 캐릭터 클래스에서 Runner.Spawn을 직접 호출하지 않는다.

6.5 호출만: 이동과 표현 API

API	용도	주의점
StartDefaultDash()	Definition의 기본 대시 실행	일반적인 대시는 이것을 사용
StartDash(direction, power, duration)	방향, 힘, 시간이 다른 대시	특수 이동 스킬에 사용
PlayActionEffect(action, position, angle)	모든 클라이언트에 행동 VFX 요청	게임 판정에는 사용하지 않음

StartDefaultDash

구현 상태는 호출만이다. 함수 내부는 이미 구현되어 있고 인자를 받지 않는다.

방향: 캐릭터가 현재 바라보는 좌우 방향

힘: Character Definition의 Default Dash Power

시간: Character Definition의 Default Dash Duration

실행 중 처리: 대시 속도를 유지하고 중력을 잠시 0으로 변경

사용 대상: 모든 캐릭터가 같은 규칙을 사용하는 일반 대시

기본 OnDash가 이 함수를 이미 호출하므로 일반 대시 캐릭터는 아래 코드를 작성할 필요가 없다.

```
// 작성하지 않아도 CharacterBase에 이 동작이 구현되어 있다.
protected override bool OnDash(CharacterActionContext context)
{
    StartDefaultDash();
    return true;
}
```

StartDash

구현 상태는 호출만이다. 방향, 힘, 시간을 캐릭터가 직접 전달하는 특수 대시용 공통 함수다.

인자	의미	내부 보정
direction	이동할 방향	자동 정규화, 0이면 바라보는 방향 사용
power	대시 속도 크기	음수이면 0으로 보정
duration	대시 유지 시간	최소 0.01초 보장

조준 방향으로 돌진하는 캐릭터라면 OnDash만 선택 구현하고 그 안에서 StartDash를 호출한다.

```
protected override bool OnDash(CharacterActionContext context)
{
    StartDash(context.AimDirection, dashPower, dashDuration);
    return true;
}
```

Q 스킬 자체가 돌진기라면 OnDash가 아니라 OnSkillQ 안에서 StartDash를 호출한다. 어떤 입력에서 실행되는지는 override 함수가 결정하고, 실제 이동 방식은 StartDefaultDash 또는 StartDash가 처리한다.

PlayActionEffect

구현 상태는 호출만이다. 모든 클라이언트에 행동 VFX를 요청하는 Presentation API다. 데미지나 이동 같은 게임 상태를 먼저 처리한 다음 결과를 보여줄 때 호출한다. 이 함수만 호출해도 데미지나 공격 판정이 생기지는 않는다.

선택 기준 요약

원하는 동작	작성할 함수	그 안에서 호출할 API
평범한 좌우 대시	아무것도 작성하지 않음	Base가 StartDefaultDash 호출
대시 버튼으로 조준 방향 돌진	OnDash override	StartDash
Q 버튼으로 조준 방향 돌진	OnSkillQ override	StartDash
대시 후 VFX 표시	OnDash override	StartDash 또는 StartDefaultDash + PlayActionEffect

6.6 조희만: 상태와 Socket

속성	의미
Definition	연결된 CharacterDefinition
Visual	연결된 CharacterVisualController
CurrentHealth	현재 체력
MaxHealth	최대 체력
CurrentHealthPercent	0부터 1 사이의 체력 비율
IsDead	사망 상태
Cooldowns	행동별 남은 쿨타임 조희 객체
Movement	지면, 대시, 속도 등 이동 상태
AimDirection	현재 동기화된 조준 방향
AimAngle	현재 동기화된 조준 각도
AttackOrigin	근접 또는 범위 공격 기준 Transform
ProjectileOrigin	투사체 생성 기준 Transform
EffectOrigin	VFX 생성 기준 Transform
WeaponRoot	무기 비주얼의 부모 Transform

쿨타임은 `Cooldowns.GetRemaining(action)`으로 남은 시간을, `Cooldowns.GetDuration(action)`으로 전체 시간을 조희할 수 있다. 기본 스킬 쿨타임 시작은 Base가 처리하므로 캐릭터 스크립트에서 다시 시작하지 않는다.

6.7 선택 구현: 패시브 이벤트 API

API	호출 시점	전달되는 값
OnPassiveTick(deltaTime)	Simulation 틱마다	틱 시간
OnDamaged(damage)	자신이 피해를 받은 뒤	요청 피해, 실제 피해, 공격자
OnDamageDealt(target, requestedDamage)	다른 캐릭터에게 피해 요청 후	대상과 요청 피해량
OnDied(attacker)	사망했을 때	마지막 공격자
OnJumped()	점프했을 때	없음
OnLanded()	착지했을 때	없음
OnSkillExecuted(action)	행동 실행에 성공했을 때	행동 종류
OnHealthChanged(previous, current)	체력이 변경됐을 때	변경 전후 체력
OnCharacterSpawned()	네트워크 캐릭터 생성 완료	없음
OnResetCharacter()	캐릭터 초기화	없음

CharacterDamageInfo.RequestedDamage는 공격자가 요청한 피해량이고, **AppliedDamage**는 체력에 실제로 반영된 피해량이다. 방어, 무적, 남은 체력의 영향까지 고려해야 하면 **AppliedDamage**를 사용한다.

이벤트로 표현할 수 있는 패시브는 **OnPassiveTick**으로 반복 검사하지 않는다. 피격 패시브는 **OnDamaged**, 착지 패시브는 **OnLanded**, 스킬 성공 패시브는 **OnSkillExecuted**에 구현한다.

7. 신규 캐릭터 개발 절차

1단계: 템플릿과 데이터 생성

1. CharacterTemplate.cs를 복사한다.
2. 클래스와 파일 이름을 캐릭터 이름으로 변경한다.
3. CharacterDefinition을 생성하고 기본 수치를 입력한다.
4. 기본 Visual Profile을 사용할지 전용 Profile이 필요한지 결정한다.

2단계: 프리팹 Variant 구성

1. 공통 Character Template Prefab의 Variant를 생성한다.
2. 캐릭터 스크립트와 Character Definition을 연결한다.
3. Body와 SpriteRenderer를 연결한다.
4. Attack Origin을 공격 기준 위치에 배치한다.
5. 필요한 경우에만 Projectile Origin, Weapon Root, Effect Origin을 연결한다.
6. CharacterVisualController에 Body와 필요한 Visual 대상을 등록한다.

3단계: 기능을 하나씩 구현

다음 순서를 권장한다.

1. 이동과 조준 확인
2. 평타 구현
3. 대시 구현 또는 기본 대시 사용
4. Q 구현
5. E 구현
6. 궁극기 구현
7. 패시브 구현
8. VFX와 사운드 연결
9. 밸런스 수치 조정

한 기능을 구현한 뒤 판정과 쿨타임을 확인하고 다음 기능으로 넘어간다. 여러 기능을 한꺼번에 작성하면 중복 실행이나 잘못된 판정의 원인을 찾기 어렵다.

4단계: 자동 구성 검사

캐릭터 프리팹을 선택한 뒤 다음 메뉴를 실행한다.

Tools > Project MS > Character > Validate Selected Character

Validator가 다음 필수 구성을 확인한다.

CharacterBase를 상속한 컴포넌트

NetworkObject

Rigidbody2D와 Collider2D

Character Definition

CharacterVisualController

5단계: Fusion 2인 테스트

Shared Mode에서 두 클라이언트를 실행하고 로컬 캐릭터와 원격 캐릭터 양쪽을 확인한다. Unity가 없는 환경에서는 코드 구조와 금지 API만 정적으로 검사하고, 이 단계는 Unity가 있는 환경에서 최종 진행한다.

8. 캐릭터 개발자가 지킬 네트워크 경계

캐릭터별 스크립트에는 원칙적으로 다음 코드를 작성하지 않는다.

Update, FixedUpdate, FixedUpdateNetwork, Render

Networked 프로퍼티

Rpc 메서드

Runner.Spawn

HasStateAuthority 또는 HasInputAuthority 검사

Rigidbody2D를 이용한 별도의 비주얼 이동

스킬 클래스는 `SpawnProjectile`, `FindEnemies...`, `DealDamage`, `StartDash`, `PlayActionEffect` 같은 공통 API를 사용한다. 공통 API만으로 구현할 수 없는 기능은 캐릭터 클래스에 Fusion 코드를 추가하지 말고 공통 시스템 담당자에게 필요한 동작을 전달한다.

공통 시스템 담당자에게 요청할 정보

기능이 시작되고 끝나는 조건

모든 클라이언트가 알아야 하는 상태

중간 입장자가 복원해야 하는 상태

생성되는 `NetworkObject`의 종류와 수명

피해가 적용되는 시점과 횟수

취소, 사망, 재접속 시 정리 규칙

충전, 채널링, 변신, 지속 장판, 설치물, 소환수처럼 여러 틱 동안 상태가 유지되는 기능은 별도 공통 네트워크 모듈이 필요할 수 있다.

9. 검수 체크리스트

프리팸과 데이터

- Character Definition이 연결되어 있는가?
- 필수 컴포넌트가 모두 있는가?
- Attack Origin이 올바른 위치에 있는가?
- 사용하지 않는 선택 Socket을 억지로 추가하지 않았는가?
- CharacterVisualController에 Body가 연결되어 있는가?

스킬과 패시브

- 한 번의 입력에 행동이 한 번만 실행되는가?
- 성공한 행동에만 쿨타임이 적용되는가?
- 데미지가 한 대상에게 중복 적용되지 않는가?
- 자기 자신이나 소유자를 공격하지 않는가?
- 패시브가 적절한 이벤트 함수에 구현되어 있는가?

네트워크

- 각 플레이어가 자기 캐릭터만 조작하는가?
- 로컬과 원격에서 공격 방향과 결과가 일치하는가?
- 투사체와 효과가 중복 생성되지 않는가?
- 체력과 사망 상태가 양쪽에서 일치하는가?
- 중간 입장자가 현재 상태를 올바르게 보는가?

비주얼

- Visual 코드가 입력을 직접 읽지 않는가?
- Visual 코드가 Rigidbody2D 상태를 변경하지 않는가?
- 점프, 착지, 피격 표현이 원격에서도 보이는가?
- 이름표와 WorldCanvas가 뒤집히지 않는가?